Instructions for JR

Two parts. **Part 1** updates the site you already have. **Part 2** puts it on a virtual server. Do Part 1 on your computer first, even if you're moving servers today.

You won't write or edit any code. You copy files, run commands, and choose a few settings.

You got	What it is	Use it when
`magicshow-final-update.zip`	Only the files that changed or are new, plus a safety check	Updating a project you already have (Part 1)
`magicshow-final-full.zip`	The whole project, every file	Starting clean, or to compare against

What's new in MagicShow final

Change	Where people find it
You stay where you are after a click or an edit. Wide tables no longer jump back to the left; Tab goes to the next box; a click right after typing works the first time	Every screen
Run of show: add a line anywhere, and drag rows up and down (mouse or finger). People and budget lines drag too	Run of show → Edit
Budget downloads as Excel (three sheets, working totals) or CSV	Budget tab
Layout downloads as a PDF, and is in the full master plan right after the Overview	Layout tab; Download PDF / Word
AI meeting summary removed from events	Gone from the event header
People create their own account with an email and password, log in with email, and can reset a forgotten password	/signup/ and the log-in page
Retainer clients can let the Blackmagic team into their workspace	Tick box at sign-up; Account in the left menu

GOOD TO KNOW

Leave these alone. The update does not touch them: `showday/settings.py`, `showday/urls.py`, `requirements.txt`, `render.yaml`, `build.sh`, `.env`. No new packages to install.

PART 1

Update the site you have (about 30 minutes)

This works from any earlier version: v2, v3, v4, Magic Maker or MagicShow.

Step 1. Back up

1. **The database.** On Render: the database page → **Recovery** → make a manual backup. On your own server with Postgres, run the command below. With SQLite, copy db.sqlite3 somewhere safe.

TYPE THIS

```
pg_dump "$DATABASE_URL" > backup-before-final.sql  ← only on your own server with Postgres
```

2. **The code.** In your project folder:

TYPE THIS

```
git status  ← should say "nothing to commit"
git checkout -b magicshow-final
```

Step 2. Unzip and run the safety check

Unzip `magicshow-final-update.zip` **next to** your project folder, not inside it:

YOUR FOLDERS

```
Projects/
   showday-prep/                ← your project
   magicshow-final-update/      ← the update you just unzipped
```

In the terminal, inside your project folder:

TYPE THIS

```
python ../magicshow-final-update/check_before_copying.py
```

If it says...	Do this
ALL CLEAR	Go to Step 3.
STOP. These code files were changed by hand	Don't copy yet. Send those files to Hananiah to merge into the update first.
STOP. You have database changes of your own	See "If you made your own database change" at the end of Part 1.

Step 3. Copy the files in

Copy **everything inside** `magicshow-final-update/showday-prep/` into your `showday-prep/` folder. Choose **Replace** for every file.

- **Windows:** open magicshow-final-update\showday-prep, press Ctrl+A then Ctrl+C, open your showday-prep folder, press Ctrl+V, choose "Replace the files in the destination".
- **Mac or Linux:** use the terminal (Finder can replace whole folders by mistake):

TYPE THIS

```
cp -R ../magicshow-final-update/showday-prep/. .
```

Step 4. Test on your computer

TYPE THIS

```
source .venv/bin/activate ← Windows: .venv\Scripts\activate
python manage.py migrate
python manage.py test
python manage.py runserver
```

Command	What you should see
`migrate`	"Applying planner.0004_final_signup_support… OK" (plus 0002 and 0003 if you didn't have them)
`test`	"Ran 41 tests" and then "OK". It takes about a minute. If anything says FAILED, stop and send the output to Hananiah.
`runserver`	Open http://127.0.0.1:8000. Press Ctrl+Shift+R once so the browser loads the new screens.

Click through these. Tick each one off:

- ☐ The log-in page says Email, with "Forgot your password?" and "Create an account" under the button
- ☐ Your old username and password still log you in
- ☐ An event's header no longer has "AI meeting summary"
- ☐ Run of show → Edit: pointing between two program rows shows "+ Add a segment here"; dragging ⠿ moves a row
- ☐ People tab: scroll the table to the right, tick a box. The table stays where you scrolled
- ☐ Budget tab: "Download Excel" saves a file that opens with Summary, Costs and Income sheets
- ☐ Layout tab: "Download layout PDF" saves a PDF with the room drawn
- ☐ Left menu: "Account" opens a pop-up with your name and email
- ☐ Log out, click "Create an account", make a test account. It opens an empty workspace of its own

Step 5. Put it online

TYPE THIS

```
git add .
git commit -m "MagicShow final"
git push -u origin magicshow-final
```

Merge the `magicshow-final` branch into main the way you normally do.

- **On Render:** merging starts a deploy that runs the database update for you. In the log, look for "Applying planner.0004… OK".
- **On your own server:** pull the new code, then:

TYPE THIS

```
python manage.py migrate
python manage.py collectstatic --no-input
sudo systemctl restart magicshow ← or however you restart the site
```

Step 6. Decide who can sign up

IMPORTANT

Read this one. Anyone who finds /signup/ can create an account, and nothing charges them yet. Until payments are connected, use a sign-up code.

Add one of these where the site's other settings live (Render: Environment; your own server: the .env file), then restart:

You want	Setting to add
Only people you give a code to (recommended for now)	`SIGNUP_CODE=pick-a-word`
Nobody. Accounts are made in /admin/ like before	`ALLOW_SIGNUP=False`
Anyone	Add nothing

With a code set, the sign-up page gets a "Sign-up code" box. Hananiah gives the code to each paying customer or retainer client. Change the word whenever you like.

Password reset emails use the same email settings as reminders (`EMAIL_HOST` and friends) and `DEFAULT_FROM_EMAIL`. If those were already set for reminders, there's nothing to add.

Step 7. Mark who is on the Blackmagic team

This is what lets you and Hananiah into a retainer client's account. "The team" means every login with Staff status ticked.

1. Go to `/admin/` → **Users**.
2. Open Hananiah's login. Tick **Staff status**. Make sure **Email address** is filled in. Save.
3. Do the same for your own login, and anyone else who helps clients.

GOOD TO KNOW

Each team login with an email also gets a short “New MagicShow account” email whenever someone signs up. Only tick Staff status for people on the team: it also lets that login open /admin/ (where they see nothing unless they’re a superuser).

How a retainer client gets connected (no admin work)

1. Send the client the site address (and the sign-up code, if you set one).
2. They click **Create an account**, fill in their name, email and a password, and tick **“I’m working with Blackmagic Media on a retainer”**.
3. Next time you or Hananiah open the app, their workspace is in the **Workspace** list in the left menu. Pick it. The menu shows a “Client workspace” tag while you’re inside.
4. Build events with them there. Set their monthly hours on the **Hours** page.

Situation	What to do
They signed up but didn’t tick the box	They open **Account** in the left menu and tick “Let the Blackmagic Media team into this workspace”. Or you tick “Blackmagic team can open this workspace” on their organization in /admin/ → Organizations.
The client wants the team out	They un-tick the same box. Access ends right away. Nothing is deleted.
Someone leaves the team	Un-tick Staff status on their login. They lose every client workspace the next time they open the app.
A regular subscriber (no retainer)	They sign up without ticking the box. Nobody on the team can see their workspace.

If something goes wrong

Problem	Fix
The site won’t start, or pages show errors	Undo the code: `git revert HEAD`, or on Render: Deploys → last good deploy → Rollback. The database doesn’t need undoing. Send Hananiah the error from the log.
“no such column: planner_organization.plan”	The database update didn’t run. Run `python manage.py migrate`.
Screens look unchanged, or buttons do nothing	The browser has old files: Ctrl+Shift+R. On your own server, run collectstatic and restart.
A team member doesn’t see a client’s workspace	Check Staff status is ticked on their login, and the box is ticked in the client’s Account. Then reload the app.
The password reset email never arrives	Check the email settings. With them blank, the email prints in the server log instead.

Problem	Fix
“Too many accounts were just created from this connection”	Sign-up allows 8 new accounts an hour from one internet address. Wait an hour, or make the account in /admin/.

If you made your own database change

The safety check stops you if planner/migrations has a file you added yourself. Copy the update in anyway (your file stays), then run:

TYPE THIS

```
python manage.py makemigrations --merge
python manage.py migrate
python manage.py test
```

PART 2

Put it on a virtual server (about 2 hours)

For Ubuntu 24.04 on any provider (DigitalOcean, Vultr, Linode, Hetzner). The same guide, with more explanation, is in the project at `docs/09-run-on-your-own-server.md`.

BEFORE YOU START

These steps follow the standard way to run a Django site and match this project's settings, but they haven't been run on your particular server. Go one part at a time and check each "should see" before moving on. Replace `magicshow.example.com` with your real address everywhere.

GOOD TO KNOW

Copying commands. A few long commands wrap onto two lines in this document. Each grey line that starts at the left edge is one command. To copy them safely, copy from the Word version of this file, or from `docs/09-run-on-your-own-server.md` in the project.

The pieces: Postgres holds the data. Gunicorn runs the Python code. Nginx faces the internet and handles https. systemd keeps Gunicorn running. cron sends the reminder emails each morning.

A. Prepare the server

First, at your domain company, add an **A record** for your subdomain pointing to the server's IP address. Then log in to the server:

TYPE THIS

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y python3-venv python3-pip git nginx postgresql certbot
python3-certbot-nginx
sudo adduser --system --group --home /srv/magicshow magicshow
sudo ufw allow OpenSSH && sudo ufw allow 'Nginx Full' && sudo ufw enable
```

The third line makes a user named magicshow that owns the site. The site never runs as root. The fourth turns on the firewall.

B. The database

TYPE THIS

```
sudo -u postgres psql
```

Inside, type these. Pick your own password, with no spaces and none of @ : / in it:

TYPE THIS INSIDE PSQL

```
CREATE USER magicshow WITH PASSWORD 'pick-a-long-password';
CREATE DATABASE magicshow OWNER magicshow;
ALTER USER magicshow CREATEDB;
\q
```

C. The code

TYPE THIS

```
sudo -u magicshow -H bash
cd /srv/magicshow
git clone https://github.com/YOUR-ACCOUNT/YOUR-REPO.git app
cd app
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt
```

No GitHub? Copy the showday-prep folder to `/srv/magicshow/app` with SFTP instead, then run the last two lines.

D. The settings file

TYPE THIS

```
cp .env.example .env
python3 -c "import secrets; print(secrets.token_urlsafe(50))" ← copy what it prints
nano .env
```

Set these lines. Leave the others as they are:

IN THE .ENV FILE

```
SECRET_KEY=paste-the-long-random-text
DEBUG=False
ALLOWED_HOSTS=magicshow.example.com
DATABASE_URL=postgres://magicshow:pick-a-long-password@localhost:5432/
magicshow
SITE_URL=https://magicshow.example.com
SECURE_SSL=False
SIGNUP_CODE=pick-a-word
EMAIL_HOST= ← your email service, with EMAIL_PORT, EMAIL_HOST_USER, EMAIL_HOST_PASSWORD
DEFAULT_FROM_EMAIL=MagicShow <no-reply@your-domain>
```

TYPE THIS

```
chmod 600 .env ← only the site can read it
```

IMPORTANT

Moving from the old server? Use the same SECRET_KEY as the old site. A new key breaks the “Mark done” links in emails already sent, and logs everyone out.

E. Build the database and the first login

TYPE THIS

```
.venv/bin/python manage.py migrate
.venv/bin/python manage.py collectstatic --no-input
.venv/bin/python manage.py createsuperuser
.venv/bin/python manage.py setup_org "Blackmagic Media" --user
THE_USERNAME_YOU_JUST_MADE
.venv/bin/python manage.py test ← should end: Ran 41 tests … OK
exit
```

Bringing your existing data over instead? On the old server:

TYPE THIS

```
pg_dump --no-owner --no-acl "$DATABASE_URL" > magicshow.sql
```

Copy that file to the new server. Then, **before** the migrate line above, run the line below. After it, run migrate, collectstatic and test, and skip createsuperuser and setup_org (your logins came over with the data). Also copy the old server’s `media/` folder, which holds uploaded files, into `/srv/magicshow/app/media/`.

TYPE THIS

```
psql "postgres://magicshow:pick-a-long-password@localhost:5432/magicshow"
< magicshow.sql
```

F. Keep the site running

TYPE THIS

```
sudo nano /etc/systemd/system/magicshow.service
```

PASTE THIS INTO THE FILE

```
[Unit]
Description=MagicShow (Gunicorn)
After=network.target postgresql.service

[Service]
User=magicshow
Group=magicshow
WorkingDirectory=/srv/magicshow/app
ExecStart=/srv/magicshow/app/.venv/bin/gunicorn showday.wsgi:application
--bind 127.0.0.1:8001 --workers 3 --timeout 60
Restart=always
RestartSec=3

[Install]
WantedBy=multi-user.target
```

TYPE THIS

```
sudo systemctl daemon-reload
sudo systemctl enable --now magicshow
sudo systemctl status magicshow --no-pager ← should say "active (running)"
```

Not running? Read the error with `sudo journalctl -u magicshow -n 50 --no-pager`.

G. Nginx and https

TYPE THIS

```
sudo nano /etc/nginx/sites-available/magicshow
```

PASTE THIS INTO THE FILE (WITH YOUR ADDRESS)

```
server {
    listen 80;
    server_name magicshow.example.com;
    client_max_body_size 25m;

    location / {
        proxy_pass http://127.0.0.1:8001;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

TYPE THIS

```
sudo ln -s /etc/nginx/sites-available/magicshow /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx
sudo certbot --nginx -d magicshow.example.com
```

Open `https://magicshow.example.com` and log in. Once that works, turn on the https-only protections: change `SECURE_SSL=False` to `SECURE_SSL=True` and restart:

TYPE THIS

```
sudo -u magicshow nano /srv/magicshow/app/.env
sudo systemctl restart magicshow
```

H. The daily reminder emails

TYPE THIS

```
sudo crontab -u magicshow -e
```

ADD THIS ONE LINE (8 AM SERVER TIME)

```
0 8 * * * cd /srv/magicshow/app && .venv/bin/python manage.py
send_reminders >> /srv/magicshow/reminders.log 2>&1
```

To see what would be sent without sending anything:

TYPE THIS

```
sudo -u magicshow bash -c "cd /srv/magicshow/app && .venv/bin/python
manage.py send_reminders --dry-run"
```

I. Backups

The data is in Postgres and uploaded files are in `/srv/magicshow/app/media/`. Turn on your host's server backups or snapshots: that covers both. The project guide (docs/09, Part I) also has a nightly database backup line for cron.

Updating the server later

TYPE THIS

```
sudo -u magicshow -H bash -c "cd /srv/magicshow/app && git pull &&
.venv/bin/pip install -r requirements.txt && .venv/bin/python manage.py
migrate && .venv/bin/python manage.py collectstatic --no-input"
sudo systemctl restart magicshow
```

If something goes wrong on the server

Problem	Where to look
502 Bad Gateway	Gunicorn isn't running: `sudo journalctl -u magicshow -n 50 --no-pager`
Bad Request (400)	ALLOWED_HOSTS in .env doesn't match the address you typed. Fix it, then restart.
The page has no styling	collectstatic didn't run. Run it, then restart.
"CSRF verification failed" when logging in	ALLOWED_HOSTS must be the exact address, and the Nginx file needs the X-Forwarded-Proto line.
"Too many redirects" after SECURE_SSL=True	The X-Forwarded-Proto line is missing from the Nginx file.
Uploads fail with "413"	The client_max_body_size line is missing from the Nginx file.
Changes to .env do nothing	Restart: `sudo systemctl restart magicshow`

Appendix: every file in the update zip

Paths are inside your showday-prep folder. The same list is in `CHANGED-FILES.txt`. Files marked NEW didn't exist in the last MagicShow version.

```
.env.example
README.md
docs/02-how-it-works.md
docs/03-put-it-online.md
docs/04-making-changes.md
docs/05-whats-next.md
docs/06-update-to-v3.md
docs/07-update-to-v4.md
docs/08-update-to-final.md   (NEW)
docs/09-run-on-your-own-server.md   (NEW)
planner/access.py
planner/accounts.py   (NEW)
planner/admin.py
planner/ai.py
planner/api.py
planner/apps.py
planner/auth.py   (NEW)
planner/budget.py
planner/checks.py
planner/emails.py
planner/ics.py
planner/management/commands/send_reminders.py
planner/migrations/0002_v3_budget_reminders_approvals.py
planner/migrations/0003_v4_workspaces_hours.py
planner/migrations/0004_final_signup_support.py   (NEW)
planner/models.py
planner/reminders.py
planner/serializers.py
planner/static/planner/css/app.css
planner/static/planner/js/account.js   (NEW)
planner/static/planner/js/actions.js
planner/static/planner/js/approvals.js
planner/static/planner/js/boot.js
planner/static/planner/js/budget.js
planner/static/planner/js/core.js
planner/static/planner/js/exports.js   (NEW)
planner/static/planner/js/helpers.js
planner/static/planner/js/hours.js
planner/static/planner/js/library.js
planner/static/planner/js/reminders.js
planner/static/planner/js/reports.js
planner/static/planner/js/store-django.js
planner/static/planner/js/templates.js
planner/static/planner/js/views.js
planner/static/planner/js/wrapup.js
planner/templates/planner/app.html
planner/templates/planner/approval_page.html
planner/templates/planner/auth/base.html   (NEW)
planner/templates/planner/auth/password_change.html   (NEW)
planner/templates/planner/auth/reset_complete.html   (NEW)
planner/templates/planner/auth/reset_confirm.html   (NEW)
planner/templates/planner/auth/reset_email.txt   (NEW)
planner/templates/planner/auth/reset_form.html   (NEW)
planner/templates/planner/auth/reset_sent.html   (NEW)
planner/templates/planner/auth/reset_subject.txt   (NEW)
planner/templates/planner/auth/signup.html   (NEW)
planner/templates/planner/done_page.html
planner/templates/planner/email.html
```

```
planner/templates/planner/public_base.html
planner/templates/planner/task_page.html
planner/templates/registration/login.html
planner/tests.py
planner/tests_final.py   (NEW)
planner/tests_v3.py
planner/tests_v4.py
planner/urls.py
planner/views.py
```